

Pulse Studio Desktop 1.0.0 · Documentación técnica

Sep 29, 2026 · @Pablo Juzgado

Pulse Studio Desktop es la versión para Windows de Pulse Studio: un backend Java con API REST local y una interfaz HTML, CSS y JavaScript que se muestra en una ventana de Microsoft Edge. Conserva las funciones de la app Android 1.6.4 y se empaqueta como `PulseStudio.exe` con `package`.

1. Alcance

La versión de escritorio hace lo mismo que la de Android: noticia, guion, seis imágenes, miniatura con IA y vídeo con subtítulos. La diferencia de fondo es dónde vive la lógica de negocio: en Android estaba en JavaScript dentro de la WebView; en escritorio está en Java y la interfaz la consume por REST.

Aspecto	Android 1.6.4	Desktop 1.0.0
Lógica de negocio (noticias, artículo, guion, fotos, IA, transcripción)	JavaScript en la WebView	Java, en el backend
Llamadas a Groq, Pexels, Hugging Face, Jina y RSS	Desde la interfaz, con las claves en memoria	Solo desde el backend. Las claves no llegan a la interfaz
Puente con el sistema	<code>PulseNative</code> (mensajes JSON)	API REST en <code>127.0.0.1</code> con token de sesión
Claves	AES-GCM con Android Keystore	AES-GCM con una clave aleatoria en un archivo con permisos solo para el usuario
Guardado	Galería (MediaStore) o selector de archivos	Carpeta de salida configurable, una subcarpeta por pack
Ventana	WebView de Android	Microsoft Edge (o Chrome) en modo aplicación
Recorte de persona y exportación de vídeo	En la interfaz (MediaPipe, Mediabunny)	Igual: siguen en la interfaz, en local
Distribución	APK	<code>PulseStudio.exe</code> (carpeta portable o instalador)

Lo que no cambia: la paleta y los componentes visuales, el asistente de cinco pasos, los prompts y esquemas JSON de Groq, los límites de la extracción de artículos y el motor de vídeo (fotograma a fotograma, MP4 a 30 fps). La interfaz se adaptó a pantalla grande y a ratón.

2. Arquitectura

La interfaz solo habla con el backend; él llama a Internet



arquitectura de Pulse Studio Desktop · 3 zonas

La interfaz se sirve desde el propio backend y solo se comunica con él, por REST y con un token de sesión. El backend hace las llamadas a Groq, Pexels, Hugging Face, Jina y RSS con las claves cifradas, y guarda los archivos en la carpeta de salida. El recorte de persona y la exportación del vídeo se hacen en la ventana, en local.

3. Stack, requisitos y estructura

El backend no tiene dependencias externas: usa solo el JDK (servidor `com.sun.net.httpserver`, `java.net.http`, `javax.crypto`, `javax.imageio`). Son 31 clases de producción y 4 clases de prueba.

Componente	Detalle
Java	17 o superior (se compila con <code>--release 17</code> ; el empaquetado se probó con JDK 21)

Componente	Detalle
Empaquetado	<code>jpackage --type app-image</code> (lleva su propio runtime). Instalador opcional con <code>--type exe</code> (requiere WiX Toolset 3.14)
Módulos del runtime	<code>java.base</code> , <code>java.desktop</code> , <code>java.logging</code> , <code>java.net.http</code> , <code>java.xml</code> , <code>jdk.httpserver</code> , <code>jdk.localedata</code> (y <code>jdk.crypto.ec</code> si el JDK lo trae)
Opciones de la JVM	<code>-Djava.net.useSystemProxies=true</code> , <code>-Dfile.encoding=UTF-8</code> , <code>-Xmx1g</code>
Ventana	Microsoft Edge (Chrome como alternativa) con <code>--app</code> . En Windows 10 y 11 Edge ya viene instalado
Interfaz	<code>index.html</code> , <code>styles.css</code> , <code>app.js</code> (JavaScript sin frameworks)
Motor local de la interfaz	MediaPipe Tasks Vision 0.10.35 y Mediabunny 1.60.0, en <code>web/vendor/</code>
Pruebas	JUnit 4.13.2 (backend) y Playwright con Chromium (extremo a extremo)

Ruta	Contenido
<code>src/main/java/com/pulsestudio/desktop/PulseStudioApp.java</code>	Punto de entrada: instancia única, servidor, ventana y cierre
<code>WindowLauncher.java</code>	Localiza y abre Edge o Chrome en modo aplicación
<code>server/</code>	<code>LocalServer</code> , <code>Router</code> , <code>Request</code> , <code>StaticFiles</code> , <code>Json</code> , <code>ApiException</code>
<code>api/</code>	Controladores REST: <code>Settings</code> , <code>News</code> , <code>Content</code> , <code>Media</code> , <code>Files</code> , <code>System</code> , y <code>Services</code> (cableado)
<code>service/</code>	Lógica de negocio: <code>News</code> , <code>Article</code> , <code>Groq</code> , <code>Script</code> , <code>Thumbnail</code> , <code>Pexels</code> , <code>HuggingFace</code> , <code>ImageCache</code> , <code>File</code> , <code>System</code> , <code>Text</code>
<code>store/</code>	<code>AppPaths</code> , <code>SecretStore</code> (claves cifradas), <code>SettingsStore</code>

Ruta	Contenido
<code>net/</code>	<code>WebClient</code> (llamadas salientes controladas) y <code>Endpoints</code> (URL base de los servicios)
<code>src/main/resources/web/</code>	La interfaz y <code>vendor/</code> (<code>MediaPipe</code> y <code>Mediabunny</code>)
<code>src/test/java/</code>	Pruebas JUnit
<code>tools/e2e/</code>	Prueba de extremo a extremo con servicios simulados y datos de prueba
<code>packaging/</code>	Icono (<code>.ico</code> y <code>.png</code>)
<code>build.bat</code> , <code>run-dev.bat</code> , <code>build.sh</code> , <code>pom.xml</code>	Compilación, desarrollo y apertura en un IDE

4. Ciclo de vida

1. **Instancia única.** Al arrancar, `PulseStudioApp` toma un bloqueo de archivo (`instance.lock`). Si ya hay otra copia abierta, lee `instance.json` (URL y token de la primera) y solo abre otra ventana contra ese backend.
2. **Backend.** Crea un token aleatorio de 32 bytes (Base64 URL), registra los controladores y arranca `LocalServer` en `127.0.0.1` con un puerto libre elegido por el sistema. Escribe `instance.json` con permisos solo para el usuario.
3. **Ventana.** `WindowLauncher` busca `msedge.exe` en `Program Files (x86)` , `Program Files` y `LOCALAPPDATA` , y luego `chrome.exe` . Lo abre con `--app=http://127.0.0.1:PUERTO/#t=TOKEN` , un perfil propio (`window-profile`), tamaño inicial `1440 × 920` y sin asistente de primera ejecución. Si no hay Edge ni Chrome, usa el navegador predeterminado.
4. **Token.** La interfaz lo lee del fragmento `#t=` , lo guarda en `sessionStorage` y lo quita de la URL. Desde entonces lo envía en la cabecera `X-Pulse-Token` .
5. **Latido.** La interfaz llama a `/api/ping` cada 10 s.
6. **Cierre.** El backend termina cuando el proceso de la ventana acaba (si duró más de 5 s) o cuando pasan 45 s sin latidos, pasados los primeros 90 s de arranque. Un vigía lo comprueba cada 15 s. Al cerrar, borra `instance.json` y libera el bloqueo.

El registro se escribe en `pulse-studio.log` (2 MB y 2 archivos rotativos). Si el arranque falla, se muestra un cuadro de diálogo con el motivo.

Opciones de desarrollo: `--no-window` (solo backend; imprime la URL con el token), `--port=N`, `--web=RUTA` (sirve la interfaz desde disco), `--data=RUTA` y `--token=VALOR` (solo pruebas).

5. API REST

Todas las rutas de `/api/` exigen la cabecera `X-Pulse-Token`. Las peticiones y respuestas son JSON, salvo los cuerpos binarios (audio, archivos, imágenes). Un error devuelve el código HTTP y `{"error": "mensaje en español"}`.

Código	Cuándo
400	Petición o parámetro no válido; falta una clave
401	Token ausente o incorrecto, o clave de un servicio rechazada
403	Host u origen no permitido
404	Ruta o servicio desconocido
405	Método no permitido
413	Archivo mayor de 1 GiB, o audio demasiado grande para Groq
502	Un servicio externo ha fallado. Si Groq alcanza su límite de solicitudes, se devuelve 429
500	Error interno

Método	Ruta	Función
GET	<code>/api/ping</code>	Latido. Devuelve <code>{ok, version}</code>
GET	<code>/api/settings</code>	Estado de cada servicio (<code>configured</code> y últimos 4 caracteres), modelos disponibles y carpeta de salida
PUT	<code>/api/settings</code>	Cuerpo <code>{keys:{groq,pexels,hf}, clear:[...], groqModel, hfModel, outputDir}</code> . Devuelve el estado actualizado
DELETE	<code>/api/settings/keys</code>	Borra todas las claves

Método	Ruta	Función
POST	/api/settings/test/{groq pexels hf}	Prueba la conexión con la clave escrita o la guardada. Cuerpo {key}
POST	/api/settings/choose-folder	Selector de carpeta de Windows; guarda la elegida
GET	/api/news? q=&category=&source=&sort=	Noticias de Hacker News y RSS. Devuelve {items, bySource, failures, partial, categoryLabel}
POST	/api/article	Cuerpo {url, title, manualText, lastExtracted, storyText, summary}. Devuelve {text, origin, automatic, characters, readerError}
POST	/api/script	Cuerpo {news, evidence, category}. Devuelve el guion y las seis búsquedas de imagen
POST	/api/photo	Cuerpo {query, exclude}. Devuelve {id, pexelsId, photographer, photographerUrl, photoUrl, query, url} con url = /api/images/{id}
GET	/api/images/{id}	JPEG 1080 × 1920 en caché
POST	/api/thumbnail/brief	Cuerpo {news, context}. Devuelve {prompt, headline, highlight}
POST	/api/thumbnail/background	Cuerpo {prompt, model}. Devuelve los bytes de la imagen
POST	/api/transcribe? name=&lang=&context=	Cuerpo: audio (audio.m4a , audio.webm o audio.mp3 , hasta 25 MB). Devuelve {words} con tiempos
POST	/api/files?name=&folder=	Cuerpo: el archivo, en streaming. Devuelve {saved, path, folder, name}

Método	Ruta	Función
POST	<code>/api/system/reveal</code>	Cuerpo <code>{path}</code> . Muestra el archivo o la carpeta en el Explorador
POST	<code>/api/system/open-url</code>	Cuerpo <code>{url}</code> . Abre un enlace en el navegador predeterminado

Desde la interfaz se usa siempre a través de `PulseApi` (`get` , `post` , `put` , `del` , `upload`), que añade el token, aplica un tiempo límite y convierte los errores en excepciones con el mensaje del backend. `PulsePlatform.saveBlob` sube los archivos a `/api/files` .

6. Lógica de negocio en el backend

Servicio	Qué hace
<code>NewsService</code>	Consulta Hacker News (Algolia, ordenado por fecha) y los RSS de HackRead, KrebsOnSecurity y Xataka a través del worker <code>/feed?source=</code> . Filtra por tema con expresiones regulares por categoría (todas, tech, ciberseguridad, IA, hardware), normaliza las URL para quitar duplicados y ordena por fecha. Analiza el RSS sin DTD ni entidades externas
<code>ArticleService</code>	Pide el texto a Jina Reader (<code>r.jina.ai</code>), quita menús, enlaces relacionados, pies de imagen, notas y líneas de ruido, y lo limita a 8 200 caracteres por párrafos. Prioridad: texto pegado por el usuario (70 caracteres o más), Jina Reader, cuerpo de la fuente RSS u Hacker News (180 o más) y, por último, el extracto de la noticia. Si no hay ninguno, pide al usuario que pegue el artículo. El límite baja a 2 900 caracteres si el texto es sobre todo chino, japonés o coreano
<code>GroqClient</code>	Chat con salida JSON de esquema estricto (<code>response_format json_schema</code>) y transcripción con <code>whisper-large-v3-turbo</code> (multipart, <code>verbose_json</code> , tiempos por palabra, temperatura 0). Traduce los errores de Groq a mensajes claros

Servicio	Qué hace
ScriptService	Construye el prompt del guion (el texto de la noticia va delimitado como datos) y el esquema: gancho, contexto, desarrollo, por qué importa, cierre y palabras clave. Compone el guion final con titular, medio, fecha y fuente, y las seis búsquedas de imagen
ThumbnailService	Pide a Groq un prompt visual en inglés, un titular de hasta 70 caracteres y una palabra destacada de hasta 30
PexelsService	Busca fotos verticales (15 por consulta), evita repetidas, descarga solo desde <code>images.pexels.com</code> y convierte cada una a JPEG 1080 × 1920 recortando al centro
ImageCache	Guarda esos JPEG con un identificador aleatorio en la carpeta de caché y los sirve por <code>/api/images/{id}</code>
HuggingFaceService	Consulta qué proveedor tiene el modelo en directo (<code>hf-inference</code> , <code>nscala</code> o <code>together</code> , por ese orden de preferencia) y genera una imagen de 768 × 1344. Si un proveedor falla, prueba el siguiente; un 401 o un 402 se devuelven de inmediato
FileService	Guarda archivos en streaming dentro de la carpeta de salida, con nombres validados y sin sobrescribir
SystemService	Abre el Explorador con el archivo seleccionado, un enlace en el navegador o el selector de carpeta

Modelos permitidos: Groq `openai/gpt-oss-20b` (por defecto) y `openai/gpt-oss-120b`; Hugging Face FLUX.1 schnell (por defecto), Stable Diffusion XL y Stable Diffusion 3 Medium.

El backend no interpreta el vídeo ni la foto del usuario: el recorte de persona, la composición y la exportación del MP4 siguen en la interfaz, en local. Solo la pista de audio, si el usuario pide subtítulos, pasa por el backend hacia Groq.

7. Seguridad

Área	Medida
Red local	El servidor solo escucha en <code>127.0.0.1</code> (bucle local), en un puerto aleatorio
Host	Se exige que la cabecera <code>Host</code> sea exactamente <code>127.0.0.1:PUERTO</code> (o <code>localhost:PUERTO</code>). Evita el <i>DNS rebinding</i>
Origen	En <code>/api/</code> se rechaza <code>Sec-Fetch-Site: cross-site</code> y cualquier <code>Origin</code> distinto del propio (403). Protege contra CSRF
Autenticación	Token de sesión aleatorio de 32 bytes, comparado en tiempo constante (<code>MessageDigest.isEqual</code>). Sin token, 401. Se pasa por el fragmento de la URL, que no se envía al servidor
Cabeceras	CSP estricta: <code>default-src 'none', script-src 'self' 'wasm-unsafe-eval', connect-src 'self', worker-src 'none', frame-ancestors 'none'</code> . Además <code>X-Content-Type-Options: nosniff</code> , <code>Referrer-Policy: no-referrer</code> , <code>X-Frame-Options: DENY</code> , <code>Cross-Origin-Opener-Policy</code> , <code>Cross-Origin-Resource-Policy</code> y <code>Permissions-Policy</code> sin cámara, micrófono ni geolocalización
Archivos estáticos	Solo rutas que cumplen <code>/[A-Za-z0-9._/-]+</code> , sin <code>..</code> , con tipo MIME conocido. Sin listados de carpeta
Claves de API	AES-256-GCM. La clave de cifrado es aleatoria, está en <code>credentials.key</code> y se protege con una ACL de solo el propietario. Los datos van en <code>credentials.aesgcm</code> (1 byte de versión, IV de 12 bytes y texto cifrado, con datos asociados <code>`PulseStudioDesktop</code>
Llamadas salientes	<code>WebClient</code> solo conecta con los hosts de <code>Endpoints</code> (lista cerrada), no sigue redirecciones y limita el tamaño de cada respuesta. Las URL base solo se pueden cambiar con propiedades de la JVM al arrancar (<code>-Dpulse.endpoint.*</code>), pensado para pruebas

Área	Medida
Archivos guardados	Nombre validado con <code>[A-Za-z0-9_-]{1,100}</code> y extensión <code>txt</code> , <code>jpg</code> , <code>jpeg</code> , <code>png</code> , <code>mp4</code> o <code>webm</code> ; carpeta con <code>[A-Za-z0-9_-]{1,80}</code> . Nunca se sobrescribe (se añade « (2)»). <code>reveal</code> solo acepta rutas dentro de la carpeta de salida
Enlaces externos	<code>open-url</code> solo admite <code>http</code> y <code>https</code>
Datos del usuario	<code>%LOCALAPPDATA%\PulseStudio</code> con permisos solo para el usuario actual (ACL en Windows, 700 y 600 en otros sistemas)
XML	El RSS se analiza con <code>disallow-doctype-decl</code> y procesamiento seguro (sin XXE)

La foto y el vídeo del usuario no salen del equipo. A Groq van el texto de la noticia y, para los subtítulos, la pista de audio; a Hugging Face, solo el prompt visual.

8. Datos, archivos y límites

Datos de la aplicación en `%LOCALAPPDATA%\PulseStudio` (en otros sistemas, `~/.pulsestudio`):

Archivo	Contenido
<code>settings.json</code>	Modelo de Groq, modelo de Hugging Face y carpeta de salida
<code>credentials.aesgcm</code> y <code>credentials.key</code>	Claves de API cifradas y clave de cifrado (archivos separados)
<code>window-profile/</code>	Perfil propio de Edge o Chrome: no mezcla historial ni extensiones con el navegador personal
<code>instance.lock</code> y <code>instance.json</code>	Instancia única: bloqueo y datos de la copia en marcha (se borra al cerrar)
<code>cache/images/</code>	JPEG del pack en curso. Se vacía al arrancar
<code>pulse-studio.log</code>	Registro, con rotación de 2 MB y 2 archivos

Los archivos que crea el usuario van a la carpeta de salida (por defecto `Documentos\PulseStudio`, configurable en Configuración). Cada pack usa su propia subcarpeta `PulseStudio_AAAAMMDD_HHMM_titular` con `guion_tiktok.txt`, `imagen_1.jpg` a `imagen_6.jpg`, `miniatura_tiktok_*.jpg` o `.png` y `pulse_reel_*.mp4`. Si un nombre ya existe, se guarda con un número (« (2) »).

Límite	Valor	Dónde se aplica
Tamaño de un archivo guardado	1 GiB	<code>FileService</code>
Audio para subtítulos	25 MB	<code>MediaController</code> (límite de Groq en el plan gratuito)
Vídeo de entrada	10 minutos y 2 GB	Interfaz (<code>MAX_VIDEO_SECONDS</code> , <code>MAX_VIDEO_INPUT</code>)
Texto del artículo	8 200 caracteres	<code>ArticleService</code>
Imagen del pack	1080 × 1920, JPEG con calidad 0,88	<code>ImageCache</code>
Fondo de miniatura de Hugging Face	768 × 1344	<code>HuggingFaceService</code>
Servidor local	Cola de 64 conexiones; un hilo por petición	<code>LocalServer</code>

El vídeo se sube a `/api/files` en streaming: el backend lo escribe a disco sin cargarlo entero en memoria.

9. Interfaz

La interfaz son tres archivos servidos por el propio backend. `app.js` tiene unas 1 280 líneas y se organiza en tres secciones:

Sección	Contenido
1. Cliente REST	<code>PulseApi</code> (<code>get</code> , <code>post</code> , <code>put</code> , <code>del</code> , <code>upload</code>) y <code>PulsePlatform</code> (<code>saveBlob</code> , <code>saveImages</code>). Lee el token de <code>#t=</code> , lo guarda en <code>sessionStorage</code> y envía el latido cada 10 s

Sección	Contenido
2. Motor multimedia	PulseMedia : MediaPipe Selfie Segmenter con la escalera de cuatro configuraciones, composición en canvas y liberación de memoria
3. Aplicación	Asistente de cinco pasos, configuración, línea de tiempo, subtítulos y exportación MP4 con Mediabunny y WebCodecs

La interfaz solo conserva presentación y cálculo local. Ya no contiene llamadas a Groq, Pexels, Hugging Face, Jina ni RSS: la CSP (`connect-src 'self'`) las impediría de todos modos.

Adaptación a escritorio

- **Misma identidad visual.** Paleta, componentes y logotipo son los de Android. Solo cambia la capa de escritorio de `styles.css` , en un bloque final con el selector `body.desktop` .
- **Distribución.** Contenedor de hasta 1 360 px. Paso 1 en dos columnas (búsqueda a la izquierda, noticias a la derecha; dos columnas de noticias desde 1 400 px). Paso 3 con guion e imágenes lado a lado. Pasos 4 y 5 con controles en dos columnas y la vista previa fija a la derecha. Configuración en tres columnas.
- **Interacción con ratón.** Cursores, estados de `hover` , foco visible con teclado, barras de desplazamiento finas, cabecera fija. En la línea de tiempo, las imágenes se reordenan arrastrando directamente con el ratón; en pantallas táctiles se mantiene la pulsación larga.
- **Configuración.** Nueva tarjeta de carpeta de salida (ruta, «Cambiar...» con el selector de Windows y «Abrir») y textos de privacidad propios de escritorio.
- **Guardado.** Tras guardar, la interfaz muestra la ruta completa y un botón « Mostrar en la carpeta» que abre el Explorador.
- **Textos.** Se sustituyeron las referencias a móvil, galería y Android por las de equipo y carpeta.

10. Compilación, empaquetado y desarrollo

Requisitos

JDK 17 o superior con `javac` , `jar` y `jpackage` (recomendado: Eclipse Temurin 21, de adoptium.net). Solo para el instalador: WiX Toolset 3.14 en el PATH. El usuario final no necesita instalar Java.

Generar el .exe

1. Descomprimir el proyecto y abrir `cmd` en su carpeta. Es mejor ejecutar `build.bat` desde `cmd` y no con doble clic, para poder leer los mensajes.
2. Ejecutar `build.bat`. El script busca un JDK (`JAVA_HOME`, `PATH` y `Program Files`), comprueba que sea de la versión 17 o superior, compila el backend, copia la interfaz dentro de las clases, crea `build\input\pulse-studio.jar` y llama a `jpackage --type app-image`.
3. Resultado: `dist\PulseStudio\PulseStudio.exe`. La carpeta entera es portable y lleva su propio runtime (alrededor de 120 MB).
4. Opcional: `build.bat exe` crea además el instalador `dist\PulseStudio-1.0.0.exe`, por usuario, con acceso directo en el escritorio y en el menú Inicio.

El `.exe` no está firmado: Windows SmartScreen avisará la primera vez. Para distribuirlo hay que firmarlo con `signtool` y un certificado de firma de código.

Desarrollo

`run-dev.bat` compila el backend y sirve la interfaz directamente desde `src\main\resources\web`; se edita y se recarga con F5, sin recompilar. `run-dev.bat --no-window --port=8080` arranca solo el backend e imprime una URL con el token, que se abre en cualquier navegador con las herramientas de desarrollo (F12). `build.sh` hace lo mismo que `build.bat` en macOS y Linux. `pom.xml` sirve para abrir el proyecto en un IDE o compilar con Maven.

Actualizar el motor local de la interfaz

MediaPipe y Mediabunny están copiados en `src/main/resources/web/vendor/`. Son los mismos archivos oficiales que usa la app Android (MediaPipe 0.10.35 y Mediabunny 1.60.0). Este proyecto no incluye un script de descarga: para actualizarlos se sustituyen los archivos a mano y se comprueba con la prueba de extremo a extremo.

11. Pruebas

Capa	Herramienta	Qué cubre
Backend	JUnit 4.13.2, 18 pruebas	<code>ArticleServiceTest</code> (7): limpieza y límite del artículo. <code>NewsServiceTest</code> (4): filtro por tema, duplicados, URL canónicas y RSS (con protección contra XXE). <code>ServerSecurityTest</code> (5): token obligatorio, origen ajeno, errores JSON, rutas estáticas, JSON. <code>StoresTest</code> (2): claves cifradas con detección de manipulación y guardado seguro de archivos
Extremo a extremo	Playwright y Chromium	Backend Java real y servicios externos simulados. Recorre los cinco pasos con <code>MediaPipe</code> real: configuración y cifrado de claves, búsqueda sin duplicados, pack con seis imágenes de 1080 × 1920, guardado de archivos en disco, miniatura, reordenación con el ratón, subtítulos y exportación a MP4 válido

La prueba de extremo a extremo también comprueba que sin token la API responde 401, que un origen ajeno recibe 403 y que la clave no vuelve a la interfaz ni aparece en claro en `credentials.aesgcm`.

Ejecución del backend con Maven: `mvn test`. Prueba de extremo a extremo (con el backend compilado y Playwright instalado): `node tools/e2e/desktop.e2e.cjs`.

Resultado en la última ejecución de este proyecto: 18 pruebas de backend correctas y prueba de extremo a extremo correcta, con un MP4 de unos 1,8 MB.

No probado: el `.exe` en un Windows real, el instalador con WiX, Edge en modo aplicación en Windows y el comportamiento con un proxy corporativo. Las pruebas se ejecutaron en Linux.

12. Limitaciones conocidas y licencias

Limitaciones

- **Ventana.** Requiere Microsoft Edge o Google Chrome (o, como último recurso, el navegador predeterminado, sin ventana de aplicación). No se usa JavaFX WebView porque le faltan WebCodecs y el soporte de WebGL2 y WebAssembly que necesitan el editor de vídeo y el recorte de persona.
- **Sin firmar.** El `.exe` no lleva firma de código y SmartScreen lo advertirá.

- **Claves.** Las claves de la app Android no se migran: hay que introducirlas de nuevo en Configuración.
- **Un usuario, un equipo.** Los datos son por usuario de Windows y el servidor solo acepta conexiones locales. No hay acceso desde otros equipos.
- **Exportación de vídeo.** La velocidad depende de la CPU y la GPU; el resultado siempre es MP4 a 30 fps. Cada vídeo admite hasta 10 minutos.
- **Servicios externos.** Groq, Pexels y Hugging Face tienen cuotas propias (por ejemplo, el plan gratuito de Hugging Face da pocos créditos al mes). Jina Reader no garantiza acceso a todos los medios.
- **Contenido generado.** El guion, el titular y el fondo de IA deben revisarse antes de publicar.

Licencias de terceros

Componente	Licencia
MediaPipe Tasks Vision 0.10.35 y modelo Selfie Segmenter (Google)	Apache 2.0
Mediabunny 1.60.0 (Vanilagy)	MPL 2.0 (bundle sin modificar, con su LICENSE en vendor/mediabunny/)
JDK empaquetado por jpackage	Según la distribución de OpenJDK usada (Temurin: GPL 2 con excepción Classpath)

El backend no usa librerías de terceros. Los servicios externos tienen sus propias condiciones de uso, y las fotos de Pexels mantienen su atribución. El proyecto no incluye claves de API.

Fuentes

Todos los datos salen del proyecto `PulseStudioDesktop` :

- `PulseStudioApp.java` y `WindowLauncher.java` (ciclo de vida y ventana).
- `server/LocalServer.java` y `server/StaticFiles.java` (seguridad del servidor y archivos estáticos); `api/*.java` (rutas y formato de las respuestas).
- `service/*.java`, `net/*.java` y `store/*.java` (lógica de negocio, llamadas salientes, cifrado y rutas de datos).
- `src/main/resources/web/` (interfaz) y `src/test/java/`, `tools/e2e/desktop.e2e.cjs` (pruebas).

- `build.bat`, `build.sh`, `run-dev.bat`, `pom.xml` y `README.md` (compilación y empaquetado).